

Logistic Regression Cheat Sheet

Step-by-step guide with formulas, code snippets, and explanations

Overview: Logistic Regression is a classification algorithm that outputs probabilities using the sigmoid function.

It models $P(y=1|x)$ using a linear combination of inputs followed by a sigmoid activation. The training process minimizes the log-loss cost function using gradient descent.

1■■■ Sigmoid Function

Converts any real value z into a probability between 0 and 1.

Formula: $\sigma(z) = 1 / (1 + e^{(-z)})$

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))  
  
# Example  
print(sigmoid(0))          # 0.5  
print(sigmoid(np.array([-2, 2]))) # [0.12, 0.88]
```

2■■■ Initialize Parameters

Creates weight vector w and bias b initialized to zero.

```
def initialize_with_zeros(dim):  
    w = np.zeros((dim, 1))  
    b = 0.0  
    return w, b
```

```
# Example  
w, b = initialize_with_zeros(3)  
print(w, b)
```

3■■■ Propagate (Forward + Backward)

Computes cost and gradients for one forward-backward pass.

Formulas: $A = \sigma(w^T X + b)$, $\text{cost} = -(1/m) \sum (Y \log A + (1-Y) \log(1-A))$

```
def propagate(w, b, X, Y):  
    m = X.shape[1]  
    A = sigmoid(np.dot(w.T, X) + b)  
    cost = -(1/m) * np.sum(Y*np.log(A) + (1-Y)*np.log(1-A))  
    dw = (1/m) * np.dot(X, (A - Y).T)  
    db = (1/m) * np.sum(A - Y)  
    return {"dw": dw, "db": db}, np.squeeze(cost)
```

4■■■ Optimize (Gradient Descent)

Updates parameters using gradients to minimize cost over multiple iterations.

```
def optimize(w, b, X, Y, num_iterations=2000, learning_rate=0.5, print_cost=False):  
    costs = []  
    for i in range(num_iterations):  
        grads, cost = propagate(w, b, X, Y)  
        w = w - learning_rate * grads["dw"]  
        b = b - learning_rate * grads["db"]  
        if i % 100 == 0:  
            costs.append(cost)
```

```

    if print_cost:
        print(f"Cost after iteration {i}: {cost:.4f}")
    return {"w": w, "b": b}, grads, costs

```

5■■■ Predict

Predicts 0 or 1 for given input using learned parameters.

```

def predict(w, b, X):
    A = sigmoid(np.dot(w.T, X) + b)
    Y_prediction = (A > 0.5).astype(int)
    return Y_prediction

```

6■■■ Model (Training + Prediction)

Combines all steps to train and evaluate logistic regression.

```

def model(X_train, Y_train, X_test, Y_test, num_iterations=2000, learning_rate=0.5, print_cost=False):
    w, b = initialize_with_zeros(X_train.shape[0])
    params, grads, costs = optimize(w, b, X_train, Y_train, num_iterations, learning_rate, print_cost)
    w, b = params["w"], params["b"]
    Y_prediction_train = predict(w, b, X_train)
    Y_prediction_test = predict(w, b, X_test)
    if print_cost:
        print(f"Train accuracy: {100 - np.mean(np.abs(Y_prediction_train - Y_train)) * 100:.2f}%")
        print(f"Test accuracy: {100 - np.mean(np.abs(Y_prediction_test - Y_test)) * 100:.2f}%")
    return {"costs": costs, "w": w, "b": b}

```

■ Data Flow Overview

```

X_train → propagate → optimize → predict → evaluate
      ↑_____↓

```

■ Study Tips

- Start by running each function separately with small arrays.
- Add `.shape` prints to understand dimensions.
- Practice explaining what happens in each step.
- Logistic Regression = one neuron → Neural Networks = many neurons stacked!